Project Sentinel: Automated Near-Earth Object Triage

> **A note on format:** this brief tells you *what* to build and *why*, including every formula and rule you need — but not the code that implements them. Each task ends with a short hint pointing at the right technique, not the answer. Writing the loops, the `try/except` blocks, and the API calls yourself is the assignment.

1. Header & Project Metadata

Field	Detail
Project Title	Project Sentinel — Automated Near-Earth Object Triage
Domain	Aerospace / Planetary Defense
Industry Mirror	The weekly triage workflow of a planetary-defense analyst: pull the live catalog of newly-tracked close-approach objects, reconcile it against internal ground-station records, and pre-flag which objects deserve a human's attention before the rest get routine review.
Required Libraries	`requests`, `json`, `csv`, `pathlib`
Explicitly Forbidden	`pandas`, `numpy` — every task below is solvable with loops, dicts, list comprehensions, and `try/except`
Deliverables	`README.md` · `notebooks/exploration.ipynb` · `src/pipeline.py` · `data/processed/clean_data.csv`

2. Phase 1: Business Understanding

Deliverable: `README.md`

Define Objectives

Build an automated pre-triage layer for a planetary-defense close-approach review. Given this week's tracked near-Earth objects (NEOs), flag the subset that combines meaningful size with a close pass, so an analyst's limited review time goes to the objects that matter most — not the full list.

Resource Audit

Resource	Detail
API access	Free NASA API key — instant signup at `api.nasa.gov` (name + email, no card required)
Rate limit	`DEMO_KEY`: 30 requests/hour, 50/day — **shared across every user on your IP address.** On a shared campus network this can be exhausted in minutes. A personal key raises this to 1,000/hour.
Data sources	NASA NeoWs `feed` endpoint (2 chained calls or more), `data/raw/ground_station_log.csv` (generated, never downloaded), 1 web scrape call

Resource	Detail
Estimated time	4–6 hours across all three phases

Target Definition (explicit math)

```
priority_watch = 1   if (max_estimated_diameter_km >= 0.14) AND
(miss_distance_lunar <= 10)
priority_watch = 0   otherwise
```

- **0.14 km (140 m)** is the size benchmark NASA's own NEO Observations Program uses as its survey-completeness goal for hazardous-size objects.
- **10 lunar distances (LD)** is a standard "notably close" cutoff in close-approach reporting (1 LD ≈ 384,400 km).
- This is **deliberately a different rule** than NASA's own `is_potentially_hazardous_asteroid` flag (defined by minimum orbit intersection distance and absolute magnitude). You are building an independent classifier, not reverse-engineering NASA's — you'll compare the two directly in the Phase 3 Validation Check.

Brainstorm Features (minimum 6)

1. `estimated_diameter_max_km`
2. `estimated_diameter_min_km`
3. `miss_distance_km`
4. `miss_distance_lunar`
5. `relative_velocity_kph`
6. `absolute_magnitude_h`
7. `num_close_approaches_in_window`
8. `confidence_score` (from the ground-station log)

ROI Framework (derived metric)

```
pct_workload_reduction = (1 - (n_flagged / n_total)) * 100
```

Compute `n_total` (every object in your cleaned pull) and `n_flagged` (`priority_watch == 1`) directly from your own dataset, then report the result as your README's headline metric: *"An analyst who only manually reviews `priority_watch == 1` objects cuts their weekly review set by X%."*

---

3. API & Data Acquisition Specifications

API Endpoint

```
GET https://api.nasa.gov/neo/rest/v1/feed
```

Parameter	Value
`start_date`	`YYYY-MM-DD`
`end_date`	`YYYY-MM-DD` — **max 7 days after `start_date`; this is a hard API limit, not a suggestion**
`api_key`	your personal key (or `DEMO_KEY` for a quick test)

A single 7-day window rarely clears 100+ objects on its own — you'll need **two chained calls or more** across two adjacent date windows, with their results merged into one list.

Hint: a loop over your two `(start_date, end_date)` tuples, calling `requests.get()` with those plus `api_key` as query params each time, and extending a running list with every object found inside `payload["near_earth_objects"]` (remember: that's a dict keyed by date, so you need `.items()` before you reach an actual object), gets you there.

Signature Messiness Warning

- `near_earth_objects` is a **dict keyed by date string**, not a flat list — you must loop over `.items()` before you ever reach an object.
- Inside every object, `estimated_diameter` values are real JSON **numbers** — but `relative_velocity` and `miss_distance` (nested one level deeper, inside `close_approach_data`) are **the exact same kind of numeric data returned as quoted strings**. Code that assumes consistent typing across one API response will crash. Every one of these needs an explicit `float()` inside a `try/except`.
- `close_approach_data` is a **list**, usually length 1 in a short window, but never index it with `[0]` unguarded — an object with no close approach in your window returns an empty list, and `[0]` on an empty list raises `IndexError`.

Log Generator Step

Extract the ids you actually pulled:

```
neo_ids = [str(obj["neo_reference_id"]) for obj in all_records]
neo_ids = list(dict.fromkeys(neo_ids))  # de-duplicate, keep first-seen order

from pathlib import Path
Path("data/raw").mkdir(parents=True, exist_ok=True)
Path("data/raw/extracted_ids.txt").write_text("\n".join(neo_ids), encoding="utf-
8")
print(f"Extracted {len(neo_ids)} unique NEO ids.")
```

Then generate the matching messy log — from the terminal:

```
python generate_sentinel_log.py --input-ids data/raw/extracted_ids.txt
```

or directly inside your notebook:

```
from generate_sentinel_log import generate_sentinel_log
generate_sentinel_log(neo_ids)
```

Either path writes `data/raw/ground_station_log.csv`, keyed on `neo_id`, with ~10% of your ids dropped and ~10% fabricated "ghost" ids injected. Your Phase 3 join must handle both directions gracefully — don't assume every API record has a log match, or that every log row has an API match.

Bonus Web Scrape Specification

Target: `https://science.nasa.gov/science-research/planetary-science/planetary-defense/near-earth-asteroids/`

This NASA Planetary Defense Coordination Office page states a running total of all near-Earth asteroids ever discovered — but not in a clean table. It's embedded in an image's descriptive text, in the form `"<N>: Total number of discovered near-Earth asteroids of all sizes."` That's realistic: real scrape targets are rarely conveniently tabular.

Fetch the page's raw HTML, find the anchor phrase `"Total number of discovered near-Earth asteroids"`, and isolate the number sitting just before it. Print whatever window of text you're working with before you trust a slice — live pages are messier up close than they look from a search snippet.

Hint: the number sits immediately before a colon, right in front of the anchor phrase — a window of roughly the 40 characters before the anchor's index is enough to isolate it once you `.strip()` and `.split()` on whitespace.

How it enriches the dataset: attach the number you extract (`total_known_neos`) as a constant reference column on every row during Dataset Packaging, and use it for one summary insight in your README: "*This week's pull represents X% of every NEO ever catalogued.*" NASA updates this page monthly, so your exact number will differ from anyone else's — that's expected, not a bug.

---

4. Phase 2: Data Understanding

Deliverable: `notebooks/exploration.ipynb`

Data Extraction & Structural Audit

Before writing any cleaning logic, write a small function that recursively walks a record — dicts within dicts within lists — and prints the type of every leaf value alongside its path, so the string-vs-number inconsistency inside `close_approach_data` becomes visible rather than assumed. Run it on 3–5 different records, not just one — you might get lucky on the first.

Hint: a function that calls itself on every value in a dict, and on the first item of any list it meets, printing `type(value)` once it hits something that isn't a dict or a list, covers this in well under fifteen lines.

Tip: `json.dumps(record, indent=2)` renders a nested record far more readably than a raw `print()` while you're exploring.

Custom EDA (native loops — no libraries)

Pick one numeric field at a time — start with `max_diameter_km` — and compute its min, max, and mean using a single pass over your cleaned list and running accumulator variables, **not** `min()`/`max()`/`sum()`. Repeat for `miss_distance_km` and `relative_velocity_kph`, remembering both need a `float()` cast first since NASA returns them as strings.

Hint: initialize your running min and max to the first value in the list before the loop starts, then update both — plus a running sum — inside the same loop.

Quality Verification

For `close_approach_data` and `absolute_magnitude_h`, count how many records are missing or empty and turn that into a percentage of your total pull. Separately, confirm `is_potentially_hazardous_asteroid` is present and boolean-typed on every record — you're about to rely on it in your Validation Check, so it's worth checking now rather than assuming.

Hint: a single loop with a running counter per field, incremented inside an `if`, gets you every one of these numbers.

---

5. Phase 3: Data Preparation

Deliverable: **`src/pipeline.py`** **&** **`data/processed/clean_data.csv`**

Cohort Filtering

Drop any record with an empty `close_approach_data` list — there's nothing to compute a distance from, so it can't be scored.

Hint: this is a one-line filter over your record list, checking the truthiness of the list itself.

Rigorous Cleaning & Imputation

Wrap every `float()` cast on `relative_velocity`/`miss_distance` values in `try/except`, and wrap your original `requests.get()` call in `try/except requests.exceptions.RequestException` too — a live API can time out mid-run and your pipeline should fail with a clear message, not a raw traceback. For the rare record missing `absolute_magnitude_h`, impute the cohort median rather than dropping the row.

Hint: `sorted(values)[len(values) // 2]` is a simplified median — it doesn't average the two middle values on an even-length list, which is fine for this exercise. A small reusable `safe_float(value, default=None)` helper will save you from repeating the same `try/except` everywhere a cast might fail.

Feature Engineering

Derive two new columns:

- `size_to_distance_ratio` = `max_diameter_km / miss_distance_lunar` (a ratio feature)

- `approach_category`, bucketed from `miss_distance_lunar`: `very_close` (≤5 LD), `close` (≤20 LD), `moderate` (≤60 LD), `distant` (>60 LD)

Then compute `priority_watch` using the Target Definition formula from Phase 1.

Hint: all three fit inside the same loop you're already using to build your cleaned records — no need for a second pass.

Dataset Packaging & Joining

Load `ground_station_log.csv` into a dict keyed by `neo_id`, then walk your cleaned records and attach whatever log fields exist for each one — using `.get()` rather than direct indexing, since ~10% of your ids won't have a match. Attach the scraped `total_known_neos` figure as a constant column on every row while you're at it.

Hint: `csv.DictReader` plus a dict comprehension builds the lookup table in one line; `dict.get()` returns `None` cleanly when a key is missing rather than raising `KeyError`.

Min-Max Scaling

Scale `size_to_distance_ratio` using:

```
scaled_x = (x - min_x) / (max_x - min_x)
```

Hint: find `min_x` and `max_x` once over the whole column first, then a single list comprehension (or loop) applies the formula to every value.

Validation Check

Cross-tabulate your `priority_watch` flag against NASA's own `is_potentially_hazardous_asteroid` in a simple 2×2 count — how often do they agree, and how often do they disagree in each direction?

Hint: a dict keyed by `(bool, bool)` tuples, with all four combinations initialized to zero and incremented as you loop through your records, is all a native-Python crosstab needs.

Your `README.md` must include one paragraph interpreting this table. The two flags measure different things on purpose, so disagreement is expected — explain *why*, using what you know about each rule's actual definition.

Finally, write your cleaned, feature-engineered, joined, and scaled records to `data/processed/clean_data.csv` with `csv.DictWriter`.

---

6. Repository Architecture & Submission Rubric

```
project_repo/
├── data/
│   ├── raw/            # extracted_ids.txt + generated ground_station_log.csv
(never hand-edited)
│   └── processed/       # clean_data.csv — your pipeline's output
```

```
├── notebooks/
│   └── exploration.ipynb
├── src/
│   └── pipeline.py       # importable AND runnable end-to-end
└── README.md
```

Category	Points	What's graded
Phase 1 `README.md` Completeness	15	Objectives, resource audit, explicit target formula, 6+ brainstormed features, ROI metric with a real computed number
Phase 2 Native EDA & Quality Audit	25	Structural audit output, native-loop min/max/mean (no libraries), null-rate and completeness figures
Phase 3 Cleaning, Imputation & Feature Engineering	25	Cohort filter justified, `try/except` type-safety throughout, semantic-vs-statistical imputation choice explained, both engineered columns present
Integration & Min-Max Scaling	15	Correct dict join against the generated log (both directions handled), scaling formula matches spec exactly, Validation Check table present and interpreted
Code Quality, PEP-8, DocStrings & Script Entry Point	20	`src/pipeline.py` runs standalone end-to-end, functions have docstrings, an `if __name__ == "__main__":` block exists
Total	**100**	